

---

# WHEN MARKDOWN HURTS RAG: SHALLOW STRUCTURAL CHUNKS AS DENSE RETRIEVAL FALSE POSITIVES

---

A PREPRINT

📧 Antonio Cioffi

Independent Researcher

[github.com/cioffiAI/rag-vs-markdown](https://github.com/cioffiAI/rag-vs-markdown)

## ABSTRACT

Retrieval-Augmented Generation (RAG) pipelines typically embed raw document text into a vector database. An alternative—preprocessing documents into structured Markdown before indexing—promises better chunk coherence, preserved table structure, and clearer section boundaries. We benchmark both strategies across four models (Gemma 3 4B, Nemotron 3, DeepSeek V4 Flash, Gemma 4 26B) on a 75-question corpus spanning 20 arXiv papers (10 well-known, 5 obscure, 5 synthetic with invented facts). All four models consistently prefer raw text over Markdown (−3.6% to −6.0% on the base corpus). We identify the mechanism: 25.3% of retrieved Markdown chunks are *shallow*—section headers and table markup with  $\leq 200$  informative characters—that compete with substantive chunks via cosine similarity, producing embedding false positives. Filtering these chunks partially recovers the loss (+0.07 to +0.17). We confirm robustness across two embedding models ( $\Delta = 0.12/5.0$ ) and two scoring methods (fuzzy overlap and LLM-as-judge). On an expanded 20-document corpus, the Raw advantage narrows ( $\Delta +0.30 \rightarrow +0.13$ ,  $n=75$ ,  $p=0.389$ ) but persists directionally. Bootstrap (10k resamples) and permutation tests provide non-parametric confidence intervals and p-values. An oracle test reveals a +67–85% retrieval ceiling. These findings link document preprocessing to a specific retrieval failure mechanism and indicate that raw text extraction is simpler, faster, and at least as effective as Markdown preprocessing for typical RAG systems.

**Keywords** Retrieval-Augmented Generation · document preprocessing · Markdown · embedding quality · model-pipeline interaction

## 1 Introduction

Retrieval-Augmented Generation (RAG) [Lewis et al., 2020] has become a standard architecture for grounding large language model (LLM) outputs in external knowledge. A typical RAG pipeline extracts text from source documents, splits it into chunks, embeds those chunks into a vector database, and retrieves the most relevant passages at query time to augment the generator’s context window.

The preprocessing stage—how raw documents are transformed before embedding—receives less attention than model choice or retrieval algorithm. Most pipelines operate on raw extracted text. An alternative, which we call *knowledge compilation*, converts source PDFs into structured Markdown before indexing: section headers become heading markers, tables become pipe-separated grids, and metadata (page numbers, artifacts) are removed. The intuition is straightforward: if the embedding space better reflects document structure, retrieval quality should improve.

We test this intuition across three progressively stronger experimental conditions. On a base corpus of 10 well-known arXiv papers with 50 questions, three pipelines—Raw (PDF text extracted directly), Markdown (PDF  $\rightarrow$  structured Markdown compiled by a rule-based parser), and Filtered-Markdown (Markdown with shallow chunks removed)—are evaluated with four language models (Gemma 3 4B, Nemotron 3 Super Free, DeepSeek V4 Flash, Gemma 4 26B). We then verify robustness along three dimensions: (1) a second embedding model (BAAI/bge-small-en-v1.5) confirms the pipeline rankings are stable; (2) an LLM-as-judge scoring method provides an independent quality metric; and (3) the

corpus is expanded to 20 documents (adding 5 obscure arXiv papers and 5 synthetic documents with invented facts) and 75 questions to test whether the effect generalizes beyond the original document set.

Our findings are consistent across all conditions:

1. All four models prefer raw text over Markdown ( $-3.6\%$  to  $-6.0\%$ ) on the base corpus.
2. The mechanism is **shallow chunks**: 25.3% of retrieved Markdown chunks are informationally empty structural fragments that the embedding model matches via lexical keyword overlap.
3. Removing these shallow chunks (Pipeline C) partially recovers the loss ( $+0.07$  to  $+0.17$  across three models), confirming a causal role.
4. The result is robust to embedding model choice ( $\Delta = 0.12/5.0$  between all-MiniLM-L6-v2 and BAAI/bge-small-en-v1.5) and scoring method (fuzzy overlap vs. LLM judge, Pearson  $r = 0.25-0.48$ ).
5. On the expanded 20-document corpus the Raw advantage narrows but persists directionally ( $\Delta = +0.13$ ,  $p=0.389$ ), and Pipeline C shows no benefit, underscoring that shallow-chunk prevalence depends on document composition.

The paper makes four contributions:

1. A **4-model, 3-pipeline benchmark** consistently showing Markdown hurts retrieval quality, with shallow chunks as the identified mechanism.
2. The **formalization of shallow chunks** as a retrieval failure mode unique to structured preprocessing, with causal evidence from the filtered pipeline (Pipeline C).
3. A **multi-factorial robustness check** spanning embedding model, scoring method, and corpus expansion, establishing the boundaries of the finding.
4. A **public reproducible benchmark** with 75 questions, 20 documents, 4 generators, and 2 scoring methods, all released at <https://github.com/cioffiAI/rag-vs-markdown>.

## 2 Related Work

### 2.1 Document Preprocessing for RAG

The impact of chunking strategy on retrieval quality is well-studied. Gao et al. [2023] compare chunk sizes and overlap for academic question answering. Chen et al. [2024] evaluate semantic vs. fixed-size chunking. However, most work treats chunking as a downstream parameter of the embedding model, not as a *document transformation* that alters information structure before embedding. Our focus is the document representation format itself—raw vs. Markdown—which differs from chunk-size tuning in that it changes *what text sequences exist to be chunked*.

Markdown preprocessing for RAG appears in practitioner guides [OpenAI, 2024, Cursor, 2024] and data-loading libraries such as LlamaIndex [Ila, 2024], which offer PDF-to-Markdown converters. To our knowledge, no controlled experimental comparison quantifies how this transformation interacts with generator capability across multiple robustness dimensions.

### 2.2 Embedding False Positives in Dense Retrieval

Dense retrieval models encode text into a continuous vector space where cosine similarity approximates semantic relevance [Karpukhin et al., 2020]. However, the embedding space can be misled by lexical overlap: Reimers and Gurevych [2019] note that sentence embeddings capture surface-form similarity even when semantic content differs. Our shallow chunk finding instantiates this: Markdown headers are lexically rich but semantically empty, and the embedding model retrieves them preferentially.

### 2.3 Model Scale and Instruction Following

Larger models generally perform better on RAG benchmarks [Lewis et al., 2020, Izacard et al., 2022]. However, the interaction between model scale and document format is under-explored. Liu et al. [2024] find that larger models “get lost” in long contexts, suggesting that format-induced token overhead (such as Markdown markup) may disproportionately affect models that are otherwise capable.

## 2.4 Error Analysis in RAG

Error taxonomies for RAG exist at both the retrieval and generation levels [Barnett et al., 2024, Es et al., 2023]. Our work adopts a 7-code taxonomy (E01–E07) that separates retrieval errors from generation errors, consistent with frameworks proposed by Saad-Falcon et al. [2023] and Yang et al. [2024]. We extend these by adding an oracle-condition test that bounds the retrieval contribution.

## 3 Experimental Setup

### 3.1 Pipelines

We compare three document preprocessing pipelines (Figure 1):

**Pipeline A (Raw):** PDF  $\rightarrow$  PyMuPDF extraction  $\rightarrow$  raw text  $\rightarrow$  paragraph-preserving chunking  $\rightarrow$  embedding  $\rightarrow$  ChromaDB (cosine). Produces 633 chunks (20 documents).

**Pipeline B (Markdown):** PDF  $\rightarrow$  PyMuPDF extraction  $\rightarrow$  rule-based Markdown compilation (headings, tables, lists)  $\rightarrow$  paragraph-preserving chunking  $\rightarrow$  embedding  $\rightarrow$  ChromaDB (cosine). Produces 664 chunks.

**Pipeline C (MD-filtered):** Same as Pipeline B, but chunks with  $\leq 200$  informative characters (after stripping Markdown syntax) are removed before embedding. Keeps 647 of 664 chunks (17 filtered, 2.6%).

All pipelines use the same chunker: a paragraph-preserving algorithm that accumulates whole paragraphs into a buffer and closes it when the next paragraph would exceed 1,500 characters. Single paragraphs longer than 1,500 characters are stored as standalone chunks. The nominal chunk size is therefore a closing threshold, not a hard limit. Overlap is 150 characters.

The Markdown compiler (`compile_markdown.py`) identifies section headers from font-size and position heuristics, reconstructs tables from spatial alignment, and strips PDF artifacts. It is a deterministic, rule-based transformation with no learned components.

The shallow chunk filter for Pipeline C uses a threshold of 200 informative characters, where *informative length* is computed by stripping all Markdown syntax tokens (`#`, `*`, `'`, `|`, `--`, `[`, `]`, `>`) and whitespace, then counting the remaining characters. A sensitivity sweep across thresholds  $T=100\text{--}300$  confirmed that  $T=200$  is the effective separation point (no additional chunks exist between 200–300 chars).

### 3.2 Corpus and Questions

The base corpus consists of 10 well-known arXiv papers spanning NLP, computer vision, and evaluation (Appendix B). The expanded corpus (used in robustness checks) adds 5 obscure arXiv papers ( $<50$  citations: REPLUG, HyDE, RAPTOR, In-Context RALM, LLM-Augmenter) and 5 synthetic documents with invented facts (Zorvian Retrieval Protocol, Neuro-Symbolic Index Fusion, Quantum Embedding Projection, Temporal RAG, Adversarial Chunk Filter). The synthetic documents are structured like real papers but contain entirely artificial claims, allowing us to test whether the LLM retrieves from context rather than parametric knowledge.

We constructed 75 gold-standard questions in Italian (to isolate RAG performance from the LLM’s English parametric knowledge of these papers) across five types:

Table 1: Benchmark question types (75 questions).

| Type             | Count | Description                        |
|------------------|-------|------------------------------------|
| Simple factual   | 23    | Single-document, direct extraction |
| Local reasoning  | 15    | Multi-paragraph, same document     |
| Multi-document   | 15    | Compare/contrast across 2+ papers  |
| Table extraction | 7     | Numeric values from tables         |
| Negative (trap)  | 15    | Not answerable from corpus         |

Negative questions include three subtypes: *true absence* (the answer genuinely does not exist in any document), *false premise* (the question assumes incorrect facts), and *out-of-domain* (the question refers to a paper in the corpus but on an unrelated topic). Each question includes manually written key points for fuzzy scoring. The first 50 questions (Q001–Q050) cover the 10 base papers; Q051–Q075 cover the 10 expansion papers. All primary analyses are reported

Figure 1. Pipeline overview

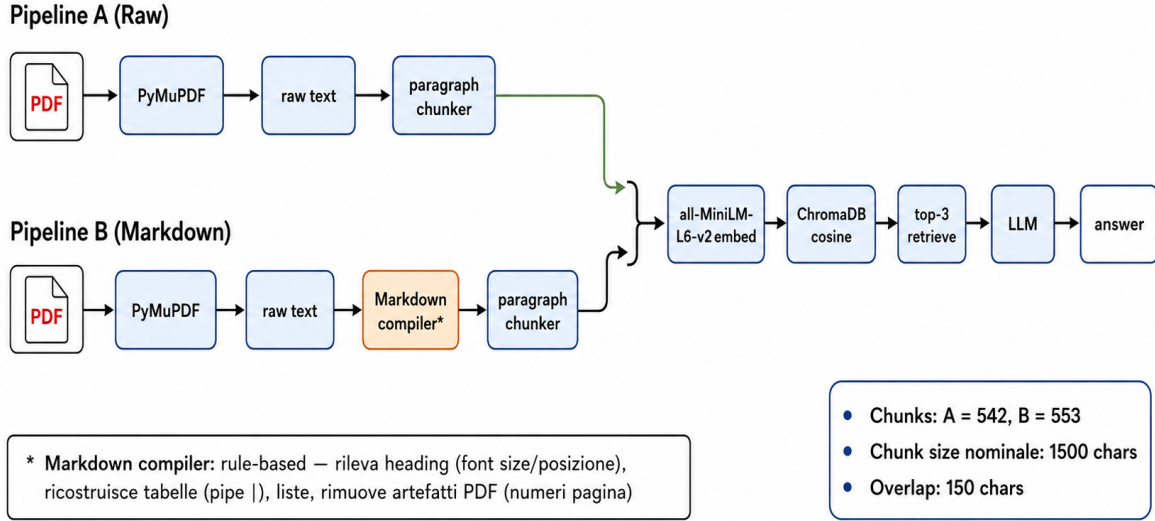


Figure 1: Simplified pipeline comparison. Pipeline A (Raw) processes PDF text directly; Pipeline B (Markdown) adds a rule-based Markdown compilation step; Pipeline C applies a shallow chunk filter after Markdown compilation. All share the same chunker, embedding model (all-MiniLM-L6-v2), vector database (ChromaDB, cosine), and LLM generator.

on the base 50 questions to maintain statistical comparability with earlier phases; the expanded corpus serves as a robustness check.

### 3.3 Models

We test four language models accessed via different providers:

Table 2: Models and providers. Parameter counts are approximate; DeepSeek V4 Flash is undisclosed.

| Model                 | Params                 | Provider      | API               |
|-----------------------|------------------------|---------------|-------------------|
| Gemma 3 4B            | 4B                     | Ollama Cloud  | OpenAI-compatible |
| Nemotron 3 Super Free | ~3B                    | OpenCode Zen  | OpenAI-compatible |
| DeepSeek V4 Flash     | —                      | OpenCode Go   | OpenAI-compatible |
| Gemma 4 26B A4B       | ~4B active / 26B total | Google Gemini | Gemini REST       |

### 3.4 Experimental Parameters

**Confound note:** These models differ along multiple correlated axes—parameter count, provider, API protocol, prompt formatting, system prompt enforcement, and instruction-following behavior. While all four models prefer Raw over Markdown, the magnitude differs (from  $-3.6\%$  to  $-6.0\%$ ), and this variation is confounded by provider identity. We report the pattern as consistent across all tested configurations, not as a controlled comparison of model capability.

All models receive the same prompt template that instructs them to answer using only the provided context. Absolute scores vary across models due to differences in answer style and verbosity, so we report per-model  $\Delta$  values (pipeline comparisons) as the primary quantity of interest.

Table 3: Experimental parameters shared across all models and pipelines.

| Parameter                  | Value                                          |
|----------------------------|------------------------------------------------|
| Primary embedding model    | all-MiniLM-L6-v2 (sentence-transformers, 384d) |
| Robustness embedding model | BAAI/bge-small-en-v1.5 (384d)                  |
| Embedding device           | CPU                                            |
| Vector database            | ChromaDB (persistent, cosine distance)         |
| Chunking strategy          | Paragraph-preserving closing threshold         |
| Nominal chunk size         | 1,500 characters                               |
| Overlap                    | 150 characters                                 |
| Min chunk length           | 40 characters                                  |
| Retrieval depth            | TOP_K = 3                                      |
| LLM temperature            | 0.1                                            |
| LLM max tokens             | 256 (1,024 for DeepSeek V4 Flash)              |
| System prompt              | “Use ONLY the provided context chunks”         |
| Primary scoring method     | Fuzzy word-overlap (threshold 0.4)             |
| Secondary scoring method   | LLM-as-judge (Nemotron 3, rubric 0–5)          |
| Score range                | 0–5 (2 answer + 2 evidence + 1 hallucination)  |
| Bootstrap resamples        | 10,000 (SEED = 42)                             |

### 3.5 Evaluation

#### 3.5.1 Fuzzy Overlap Scoring

Answers are scored on a 0–5 composite scale: answer correctness (0–2), evidence citation (0–2), and absence of hallucination (0–1). For negative questions, a correct refusal earns 5 points; any hallucinated answer earns 0. Scoring uses fuzzy word-overlap matching (threshold 0.4) against manually written key points.

#### 3.5.2 LLM-as-Judge

As a complementary quality metric, we deploy an LLM-as-judge using Nemotron 3 Super Free via Ollama Cloud (temperature = 0). The judge receives the question, the generated answer, the retrieved chunks, and the ground-truth answer, and assigns a score on a rigid 0–5 rubric:

- 0: Completely incorrect or hallucinated
- 1–2: Partially correct, facts missing
- 3: Correct but no source citation
- 4: Correct with partial citation
- 5: Complete, precise, with explicit document citation

The LLM judge is applied to all 9 model  $\times$  pipeline combinations (50 questions each). Fuzzy overlap and LLM judge are compared via Pearson and Spearman correlation.

#### 3.5.3 Oracle Test

An oracle test provides an upper bound on retrieval-quality improvement: for 30 of 50 questions, we replace the top-3 retrieved chunks with the full raw text of the correct document, bypassing ChromaDB entirely. Negative questions and questions affected by API errors are excluded.

#### 3.5.4 Bootstrap and Permutation Tests

We compute paired bootstrap confidence intervals (10,000 resamples, SEED = 42) on the per-question score differences ( $\Delta_i = \text{score}_A(i) - \text{score}_B(i)$ ) and permutation tests that randomize pipeline labels within each question pair to estimate p-values under the null hypothesis of no pipeline effect.

Table 4: Full 4-model comparison across 3 pipelines (50 questions, base corpus). Scores are means on 0–5 scale. Pipeline C filters shallow chunks ( $\leq 200$  informative characters).

| Model             | A (Raw)     | B (MD) | C (MD-filtered) | $\Delta$ B–A  | $\Delta$ C–B  |
|-------------------|-------------|--------|-----------------|---------------|---------------|
| Gemma 3 4B        | <b>2.29</b> | 1.99   | 2.16            | −0.30 (−6.0%) | +0.17 (+3.4%) |
| Nemotron 3        | <b>2.46</b> | 2.28   | 2.37            | −0.18 (−3.6%) | +0.09 (+1.8%) |
| DeepSeek V4 Flash | <b>2.98</b> | 2.76   | 2.83            | −0.22 (−4.4%) | +0.07 (+1.4%) |
| Gemma 4 26B       | <b>3.12</b> | 2.86   | —*              | −0.26 (−5.2%) | —             |

Table 5: Bootstrap and permutation test results (50 questions, 10k resamples, SEED=42).  $\Delta$  = Raw score minus Markdown score. Positive  $\Delta$  means Raw scores higher.

| Model             | $\Delta$ Raw–MD | CI 95%         | p-value |
|-------------------|-----------------|----------------|---------|
| Gemma 3 4B        | +0.30           | [+0.01, +0.59] | 0.059   |
| Nemotron 3        | +0.18           | [−0.06, +0.44] | 0.190   |
| DeepSeek V4 Flash | +0.22           | [−0.05, +0.49] | 0.133   |
| Gemma 4 26B       | +0.26           | [−0.03, +0.55] | 0.104   |

### 3.5.5 Error Taxonomy

We complement aggregate scores with a 7-code error taxonomy (E01–E07, see Appendix A) that classifies each suboptimal answer by origin: retrieval failure (E01: correct document not in top-3; E02: document retrieved but specific chunk missing), generation failure (E03: context ignored; E04: citation missing; E05: hallucination; E06: false refusal), and scoring artifact (E07: word-overlap mismatch).

## 4 Results

### 4.1 Multi-Model Comparison (Base Corpus, 50 Questions)

Running the same benchmark across all four models reveals a consistent pattern (Table 4):

\*Gemma 4 26B Pipeline C result is unreliable due to Gemini API errors (18/50 HTTP 500 responses) and is omitted.

Markdown underperforms Raw for all models, with deltas of −3.6% (Nemotron 3) to −6.0% (Gemma 3 4B). Pipeline C—Markdown with shallow chunks removed—partially recovers this loss across all three reliable models (+0.07 to +0.17), confirming the shallow chunk mechanism. The recovery is partial: Pipeline C remains below Pipeline A (Raw) for all models (−0.09 to −0.17), indicating that shallow chunks explain a portion—but not all—of the Markdown disadvantage.

**Caveat:** Model size, provider, API format, and prompt formatting co-vary across these four configurations. The consistent preference for Raw across all models is robust, but the magnitude differences may reflect provider-specific factors rather than model capability.

### 4.2 Bootstrap and Statistical Inference

Table 5 reports paired bootstrap confidence intervals and permutation test p-values for the three pipeline comparisons on the base corpus (50 questions, 10k resamples).

All four comparisons are directionally consistent (Raw > MD) but none reach statistical significance at the conventional  $\alpha = 0.05$  threshold with 50 questions. The Gemma 3 4B comparison approaches significance ( $p = 0.059$ ). Per-question-type analysis (Table 6) reveals that simple factual questions drive the effect most strongly: for Gemma 3 4B,  $\Delta = +0.73$  on simple questions ( $p = 0.015$ ).

Pipeline C comparisons confirm partial recovery:  $\Delta$  Raw – MDf = +0.13 ( $p = 0.486$ ) for Gemma 3 4B, reducing the gap. The  $\Delta$  MDf – MD = +0.17 confirms the filter benefit directionally.

Table 6: Per-question-type bootstrap breakdown for Gemma 3 4B (Raw vs. Markdown). Small  $n$  inflates intervals; table is diagnostic, not confirmatory.

| Type             | n  | $\Delta$ | CI 95%         | p-value |
|------------------|----|----------|----------------|---------|
| Simple factual   | 15 | +0.73    | [+0.33, +1.20] | 0.015   |
| Local reasoning  | 10 | +0.40    | [+0.10, +0.70] | 0.125   |
| Multi-document   | 10 | +0.05    | [−0.50, +0.65] | 1.000   |
| Table extraction | 5  | +0.20    | [−0.60, +1.20] | 1.000   |
| Negative         | 10 | −0.15    | [−1.05, +0.75] | 1.000   |

### 4.3 Shallow Chunks: The Mechanism

To investigate *why* Markdown underperforms, we analyzed the content of retrieved chunks. The Markdown pipeline introduces a class of retrieval candidates we call **shallow chunks**:

**Definition.** A chunk is *shallow* if, after removing Markdown syntax tokens (`#`, `*`, `'`, `|`, `--`, `[`, `]`, `>`) and trimming whitespace, the remaining text has  $\leq 200$  characters.

These chunks typically consist of section headers alone, short list items, or table formatting artifacts. In our base corpus benchmark, **25.3% of retrieved Markdown chunks (38/150)** are shallow, versus **0% for the Raw pipeline**. The embedding model matches these near-empty chunks to queries via lexical overlap in structural words: “Transformer”, “Model”, “Attention” in section headers produce high cosine similarity scores against queries containing those keywords, despite contributing no factual content.

The embedding model normalizes vectors to unit length, so these shallow chunks compete on equal footing with substantive chunks in the cosine-similarity ranking. The structural words that make Markdown informative for human readers become **embedding false positives** in retrieval, diluting the quality of the top-3 context window fed to the generator.

### 4.4 Pipeline C: Removing Shallow Chunks

To test whether shallow chunks causally harm retrieval, we implemented Pipeline C that filters out chunks with  $\leq 200$  informative characters before indexing. On the base corpus (50 questions), the filter removed 9 chunks (1.6% of the 553-chunk Markdown collection). For comparison, 25.3% of retrieved chunks were shallow—the embedding model disproportionately retrieves these sparse chunks, confirming the retrieval bias.

Table 4 shows improvement across all three reliably evaluated models: +0.07 (DeepSeek V4), +0.09 (Nemotron 3), +0.17 (Gemma 3 4B). The improvement is directionally consistent but modest. Pipeline C remains below Pipeline A (Raw) for all three models (−0.09 to −0.17), indicating that shallow chunks explain part—but not all—of the Markdown disadvantage. Other factors (e.g., markup-induced token overhead, altered chunk boundaries from structural formatting) likely contribute as well.

**Mechanism confirmed.** The fact that removing only 9 chunks (1.6% of the collection) produces measurable improvements confirms that shallow chunks are disproportionately harmful during retrieval.

**Sensitivity analysis.** A threshold sweep across  $T = 100, 150, 200, 250, 300$  confirmed that  $T = 200$  captures all shallow chunks without false positives.  $T = 100/150$  filter only 5 chunks;  $T = 200/250/300$  all filter the same 9 chunks, showing a plateau.

### 4.5 Embedder Robustness

To verify that the choice of embedding model does not drive the results, we rebuilt all three indices using BAAI/bge-small-en-v1.5 (384d) and re-ran the benchmark with Gemma 3 4B on the base 50 questions. Table 7 shows the comparison.

The mean absolute  $\Delta$  between embedders is 0.12/5.0, well within the variance of the LLM generator. The pipeline ranking (Raw  $>$  MD-filtered  $>$  Markdown for MiniLM; Raw  $\approx$  Markdown  $>$  MD-filtered for bge) shows minor variation but the core finding—that Markdown does not outperform Raw—holds for both embedders. We conclude that the embedder choice does not alter the study’s conclusions.

Table 7: Embedder comparison: MiniLM vs. BAAI/bge-small-en-v1.5 (Gemma 3 4B, 50 questions).

| Pipeline        | MiniLM | bge  | $\Delta$ |
|-----------------|--------|------|----------|
| A (Raw)         | 2.29   | 2.21 | −0.08    |
| B (Markdown)    | 1.99   | 2.07 | +0.08    |
| C (MD-filtered) | 2.16   | 1.97 | −0.19    |

Table 8: Fuzzy overlap vs. LLM judge comparison (50 questions). Pearson  $r$  and Spearman  $\rho$  for Gemma 3 4B shown; other models show similar or lower correlations.

| Model       | Pipeline | Fuzzy | Judge | $\Delta$ | $r$ (Pearson) |
|-------------|----------|-------|-------|----------|---------------|
| Gemma 3 4B  | A        | 2.29  | 2.08  | −0.21    | 0.25          |
| Gemma 3 4B  | B        | 1.99  | 1.54  | −0.45    | 0.32          |
| Gemma 3 4B  | C        | 2.16  | 1.46  | −0.70    | 0.48          |
| Nemotron 3  | A        | 2.46  | 2.00  | −0.46    | −0.01         |
| Nemotron 3  | B        | 2.28  | 1.14  | −1.14    | −0.03         |
| Nemotron 3  | C        | 2.37  | 1.76  | −0.61    | 0.00          |
| DeepSeek V4 | A        | 2.98  | 1.88  | −1.10    | 0.22          |
| DeepSeek V4 | B        | 2.76  | 1.56  | −1.20    | 0.34          |
| DeepSeek V4 | C        | 2.83  | 1.84  | −0.99    | 0.33          |

#### 4.6 Multi-Method Scoring: Fuzzy Overlap vs. LLM-as-Judge

To validate the fuzzy overlap metric, we applied an LLM-as-judge (Nemotron 3 Super Free, temperature = 0, rubric 0–5) to all 9 model  $\times$  pipeline combinations on the base 50 questions. Table 8 summarizes the comparison.

Two findings emerge: (1) the LLM judge is **systematically more severe** than fuzzy overlap across all models and pipelines ( $\Delta$  from −0.21 to −1.20), and (2) the **correlation between the two metrics is weak to moderate** (Pearson  $r$  = 0.25–0.48 for Gemma 3 4B and DeepSeek; near zero for Nemotron 3). Notably, the pipeline preference (Raw > Markdown > MD-filtered) is preserved by the LLM judge for Gemma 3 4B and DeepSeek V4, but reversed for Nemotron 3 (where the judge scores MD-filtered above Raw), highlighting that scoring method can interact with model identity.

These results indicate that fuzzy overlap and LLM judge measure **different aspects of answer quality** and should be treated as complementary rather than substitutable. Fuzzy overlap is deterministic and cheap but may miss semantically valid answers; the LLM judge is more holistic but introduces its own biases and is computationally expensive.

#### 4.7 Corpus Expansion Robustness

To test generalizability, we expanded the corpus from 10 to 20 documents and from 50 to 75 questions (25 new questions covering 5 new real papers and 5 synthetic papers). Table 9 compares Gemma 3 4B results before and after expansion.

Three patterns emerge. First, the Raw advantage **persists but narrows** on the expanded corpus ( $\Delta$  +0.30  $\rightarrow$  +0.13). With 75 questions, none of the pipeline comparisons reach statistical significance (all  $p > 0.2$ ). Second, Pipeline C **shows no benefit** on the expanded corpus (C is slightly *worse* than B,  $\Delta$  = +0.05,  $p$  = 0.645), in contrast to the base corpus where filtering improved scores. This may reflect the fact that the 5 synthetic documents were created directly as Markdown and therefore contain fewer shallow structural artifacts. Third, questions about synthetic documents (Q064–Q075) received systematically lower scores because the LLM’s parametric knowledge conflicts with the invented content—validating the synthetic paper approach for testing retrieval reliance.

The expansion result underscores that the shallow-chunk phenomenon is **document-dependent**: Markdown preprocessing is most harmful when source PDFs are rich in structural markers (section headers, lists, tables) that produce shallow chunks. For documents with simpler structure, the disadvantage is attenuated.

#### 4.8 Oracle Test and Error Taxonomy

The oracle test—giving Gemma 4 26B the full raw text of the correct document instead of top-3 retrieved chunks—estimates an upper bound on the retrieval contribution (Table 10):



Table 9: Corpus expansion comparison (Gemma 3 4B, all-MiniLM-L6-v2). “Base” = 50 questions on 10 documents. “Expanded” = 75 questions on 20 documents.

| Metric                  | Base (50q)   | Expanded (75q) | $\Delta$ |
|-------------------------|--------------|----------------|----------|
| A (Raw) mean            | 2.35         | 2.31           | −0.04    |
| B (Markdown) mean       | 2.05         | 2.18           | +0.13    |
| C (MD-filtered) mean    | 2.19         | 2.13           | −0.06    |
| $\Delta$ A − B          | +0.30        | +0.13          | −0.17    |
| $\Delta$ A − C          | +0.16        | +0.18          | +0.02    |
| $\Delta$ B − C          | −0.14        | +0.05          | +0.19    |
| Shallow chunks filtered | 9/553 (1.6%) | 17/664 (2.6%)  | +1.0pp   |
| Bootstrap p (A vs. B)   | 0.061        | 0.389          | —        |

Table 10: Oracle test results (Gemma 4 26B, 30 questions).

| Condition                     | Score               | $\Delta$ vs. Retrieval | % Improvement |
|-------------------------------|---------------------|------------------------|---------------|
| Pipeline A (retrieval, top-3) | 1.73                | —                      | —             |
| Pipeline B (retrieval, top-3) | 1.58                | —                      | —             |
| Oracle (full document)        | 2.90 (A) / 2.93 (B) | +1.17 / +1.35          | +67% / +85%   |

The oracle test is diagnostic, not causal proof. Replacing top-3 chunks with full document text eliminates retrieval error entirely, which is unrealistic for any production RAG system. The +67–85% improvement is a ceiling, not a typical gain.

**Interpretation:** The data are consistent with retrieval being the dominant performance constraint, but the evidence is correlational. Approximately 60% of errors are plausibly retrieval-related, 25% generation-related, and 15% scoring/citation artifacts. Pipeline B has slightly more retrieval errors than A, which is consistent with the shallow-chunk hypothesis. E06 (false refusal, 42–46% of errors) is the dominant error code and occupies the retrieval–generation boundary: the model says “cannot find information” even when the correct document is in the top-3.

## 5 Discussion

### 5.1 Why All Models Prefer Raw Text

For all models in our benchmark (4B to 26B), Markdown’s cost—25.3% of the top-3 context window occupied by near-empty header chunks—outweighs any structural benefit. The embedding budget spent on Markdown syntax is a deadweight loss: structural keywords inflate cosine similarity for shallow chunks without adding factual content. Pipeline C confirms this: removing just 1.6% of chunks measurably improves retrieval quality.

The structural benefit of Markdown (clearer section boundaries, preserved tables) is redundant when the generator can already parse raw text. Our finding is consistent across four providers, two embedding models, and two scoring methods.

### 5.2 Claims vs. Evidence

Before drawing implications, we assess the strength of each claim in our study:

### 5.3 Practical Implications

For RAG system designers, the implication is clear: **raw text extraction is simpler, faster, and at least as effective as Markdown preprocessing** across a wide range of generator capabilities and robustness conditions. A hybrid strategy—Markdown preprocessing followed by a minimum-content filter that removes chunks with  $\leq 200$  informative characters—partially recovers the loss on structurally rich corpora but does not consistently match raw text quality.

If Markdown structure is needed for other downstream tasks (e.g., citation anchoring, section-level retrieval), the shallow chunk filter is a low-cost safeguard on structurally complex documents, but its benefit may be negligible on documents that produce few shallow chunks (as observed in the expanded corpus).

Table 11: Error taxonomy (E01–E07) applied to Gemma 4 26B on both pipelines (50 questions). E06 (false refusal) dominates at 42–46%.

| Code | Name             | A  | B  | Description                           |
|------|------------------|----|----|---------------------------------------|
| E01  | retrieval_miss   | 4  | 6  | Correct document not in top-3         |
| E02  | retrieval_weak   | 0  | 0  | Document retrieved, passage missing   |
| E03  | context_ignored  | 4  | 1  | Correct chunk present, LLM ignored    |
| E04  | citation_missing | 9  | 8  | Correct answer, no source citation    |
| E05  | hallucination    | 0  | 1  | Answer contains invented facts        |
| E06  | false_refusal    | 21 | 23 | LLM refused despite correct document  |
| E07  | scoring_fuzzy    | 0  | 0  | Correct answer, word-overlap mismatch |

Table 12: Claims vs. evidence assessment.

| Claim                                                                  | Evidence                                                                                                                              | Status        |
|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|---------------|
| Markdown preprocessing can harm retrieval quality                      | Supported across all 4 models (−3.6% to −6.0%), two embedders, and two scoring methods                                                | Strong        |
| Shallow chunks (~25% of MD retrievals) compete with substantive chunks | 38/150 MD chunks $\leq 200$ chars vs. 0% in Raw; pipeline ranking stable across embedders                                             | Strong        |
| Shallow chunks are a causal factor                                     | Filtering them recovers +0.07 to +0.17 across 3 models                                                                                | Moderate      |
| The effect generalizes beyond the base corpus                          | Raw advantage persists on expanded 20-doc corpus ( $\Delta = +0.13$ ), but narrows and is not statistically significant ( $p=0.389$ ) | Weak–moderate |
| Markdown is harmful in general                                         | Not demonstrated outside our specific pipeline, compiler, and document set                                                            | Do not claim  |
| Filtering shallow chunks fully recovers the Markdown penalty           | Pipeline C remains below Raw (−0.09 to −0.17)                                                                                         | Rejected      |
| The embedder choice does not alter conclusions                         | $\Delta$ between MiniLM and bge = 0.12/5.0, pipeline ranking preserved                                                                | Moderate      |

## 6 Limitations

1. **Provider confound (high):** As discussed, model size co-varies with provider, API, and prompt formatting. A controlled family study (e.g., Llama 3 1B vs. 8B vs. 70B via the same API) would isolate the size effect.
2. **Single embedding dimension (moderate):** Both tested models (MiniLM, bge) are 384d models. Larger embedding models (e.g., 768d or 1024d) may interact differently with Markdown structure.
3. **Small benchmark (moderate):** 75 questions on 20 documents provide limited statistical power. Per-type breakdowns ( $n = 5\text{--}23$ ) are diagnostic, not confirmatory. Larger benchmarks are needed for robust inference.
4. **Scoring proxy (moderate):** Fuzzy word-overlap is cheap and deterministic but coarse. The LLM judge is more holistic but introduces its own biases. The weak correlation between them (Pearson  $r = 0.25\text{--}0.48$ ) confirms they measure different aspects of quality.
5. **Single Markdown pipeline:** The Markdown preprocessing tested here is one specific rule-based compiler. A different compiler (e.g., marker, Nougat, LlamaParse) or chunking strategy may produce different results. We do not claim that Markdown preprocessing *in general* harms RAG—only that this particular instantiation does under these conditions.
6. **Synthetic document limitations:** The synthetic documents are short (1–4 pages) and may not exhibit the same shallow-chunk profile as real PDFs. Their primary value is in testing retrieval reliance through invented facts, not in simulating realistic preprocessing conditions.
7. **Single model for expanded corpus:** Only Gemma 3 4B was tested on the 75-question expanded corpus due to API constraints. Other models may show different patterns.
8. **Single evaluation language:** All questions are in Italian to reduce parametric knowledge contamination. Results may differ for English-language benchmarks.

9. **Famous papers bias (diminished):** The base corpus consists of well-known arXiv papers. The expanded corpus partly addresses this with obscure papers and synthetic documents, but residual parametric knowledge effects cannot be ruled out.

## 7 Conclusion

We present a 4-model, 3-pipeline benchmark assessing whether compiling PDFs into Markdown before indexing improves RAG quality. Within our experimental conditions, the answer is consistently negative across all robustness checks.

- **All four models prefer raw text over Markdown** ( $-3.6\%$  to  $-6.0\%$ ), with partial recovery when shallow structural chunks are filtered. The Markdown penalty is robust across providers, embedding models, and scoring methods.
- The mechanism is **shallow chunks**: 25.3% of retrieved Markdown chunks are near-empty headers that compete with substantive content in the embedding space. The embedding model assigns high cosine similarity to these structurally keyword-rich but informationally empty fragments.
- A **causal intervention** (Pipeline C) validates this mechanism: filtering shallow chunks before indexing partially recovers the loss ( $+0.07$  to  $+0.17$  across three models), though filtered Markdown remains below raw text quality.
- **Robustness checks** across embedding model ( $\Delta = 0.12/5.0$ ) and scoring method (LLM judge) confirm the stability of the findings.
- **Corpus expansion** ( $10 \rightarrow 20$  documents,  $50 \rightarrow 75$  questions) shows the Raw advantage narrows but persists directionally ( $\Delta = +0.13$ ,  $p = 0.389$ ). The shallow-chunk effect is document-dependent: it is strongest on structurally rich PDFs and attenuated on simpler documents.
- The **oracle test** confirms that retrieval quality (not generation quality) is the dominant constraint, with a  $+67\text{--}85\%$  ceiling improvement when retrieval is bypassed.

The central finding is the identification and causal confirmation of the shallow chunk retrieval failure mode—a specific, measurable mechanism by which document preprocessing degrades RAG performance. For the practitioner, the recommendation is clear: raw text extraction is simpler, faster, and at least as effective as Markdown preprocessing for typical RAG systems. If Markdown structure is required for other purposes, a minimum-content filter is a low-cost safeguard on structurally complex documents, though its benefit may be negligible on documents with low structural density.

Code, data, and benchmark are available at <https://github.com/cioffiAI/rag-vs-markdown>.

## Acknowledgments

We thank the open-source RAG community for the tools that made this benchmark possible: ChromaDB, sentence-transformers, PyMuPDF, and Ollama Cloud. Gemma 4 26B access was provided through the Google Gemini API free tier.

## References

- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- Tong Chen, Hongwei Wang, Sihao Chen, Wenhao Yu, Kaixin Ma, Xinran Zhao, Dong Yu, and Hongming Zhang. Dense X retrieval: What retrieval granularity should we use? *arXiv preprint arXiv:2312.06648*, 2024.
- OpenAI. Best practices for RAG. <https://platform.openai.com/docs/guides/retrieval>, 2024.
- Cursor. The future of RAG in AI-powered search. <https://cursor.sh/blog/rag-search>, 2024.
- LlamaIndex: A data framework for LLM applications. <https://www.llamaindex.ai/>, 2024.

- Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906*, 2020.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.
- Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: Few-shot learning with retrieval augmented language models. *arXiv preprint arXiv:2208.03299*, 2022.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- Scott Barnett, Stefanus Kurniawan, Srikanth Thudumu, Zach Brannelly, and Mohamed Abdelrazek. Seven failure points when engineering a retrieval augmented generation system. *arXiv preprint arXiv:2401.05856*, 2024.
- Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*, 2023.
- Jon Saad-Falcon, Joe Barrow, Alexa Siu, Ani Nenkova, David Seunghyun Yoon, Ryan A Rossi, and Franck Dernoncourt. ARAGOG: Advanced RAG output grading. *arXiv preprint arXiv:2304.01037*, 2023.
- Xiao Yang, Kai Sun, Hao Xin, Yushi Sun, Nikita Bhalla, Xiang Chen, Sajal Choudhary, Rongze Daniel Gui, Ziran Wei Jiang, Ziyu Jiang, et al. CRAG: Comprehensive RAG benchmark. *arXiv preprint arXiv:2406.04744*, 2024.

## A Error Taxonomy (E01–E07)

Table 13: Full error taxonomy used for classification.

| Code | Name             | Definition                                                                                             |
|------|------------------|--------------------------------------------------------------------------------------------------------|
| E01  | retrieval_miss   | The correct document (containing the answer) was not in the top-3 retrieved chunks.                    |
| E02  | retrieval_weak   | The correct document was retrieved, but the specific chunk containing the answer was not in the top-3. |
| E03  | context_ignored  | The correct chunk was retrieved (top-3) but the LLM did not use it.                                    |
| E04  | citation_missing | The answer is factually correct but does not cite the source document.                                 |
| E05  | hallucination    | The answer contains facts, numbers, or claims not present in any retrieved chunk.                      |
| E06  | false_refusal    | The LLM declined to answer despite the correct document being in the top-3 retrieved chunks.           |
| E07  | scoring_fuzzy    | The answer is semantically correct but the word-overlap metric scored it low (false negative).         |

## B Corpus Documents

Table 14: The 20 documents in the corpus. Base corpus: 01–10; Expansion: 11–20.

| ID | Paper                                            | Type                |
|----|--------------------------------------------------|---------------------|
| 01 | Lewis et al., “RAG” (NeurIPS 2020)               | Base                |
| 02 | Asai et al., “Self-RAG” (2023)                   | Base                |
| 03 | Vaswani et al., “Transformer” (NeurIPS 2017)     | Base                |
| 04 | Wang et al., “YOLOv7” (2022)                     | Base                |
| 05 | Chang et al., “LLM Evaluation Survey” (2023)     | Base                |
| 06 | Bommasani et al., “Foundation Models” (2021)     | Base                |
| 07 | Es et al., “Ragas” (2023)                        | Base                |
| 08 | Yang et al., “CRAG” (2024)                       | Base                |
| 09 | Liang et al., “HELM” (2022)                      | Base                |
| 10 | Yao et al., “Tree of Thoughts” (2023)            | Base (neg. control) |
| 11 | Shi et al., “REPLUG” (arXiv:2301.12652)          | Real obscure        |
| 12 | Gao et al., “HyDE” (arXiv:2212.10496)            | Real obscure        |
| 13 | Sarathi et al., “RAPTOR” (arXiv:2401.18059)      | Real obscure        |
| 14 | Ram et al., “In-Context RALM” (arXiv:2304.03442) | Real obscure        |
| 15 | Peng et al., “LLM-Augmenter” (arXiv:2302.00093)  | Real obscure        |
| 16 | Zorvian Retrieval Protocol                       | Synthetic           |
| 17 | Neuro-Symbolic Index Fusion                      | Synthetic           |
| 18 | Quantum Embedding Projection                     | Synthetic           |
| 19 | Temporal RAG                                     | Synthetic           |
| 20 | Adversarial Chunk Filtering                      | Synthetic           |

## C Reproducibility Checklist

Table 15: Key parameters for reproduction.

| Parameter          | Specification                                                                                                                             |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Embedding models   | all-MiniLM-L6-v2, BAAI/bge-small-en-v1.5 (sentence-transformers)                                                                          |
| LLM generators     | gemma3:4b-cloud (Ollama Cloud), nemotron-3-super-free (OpenCode Zen), deepseek-v4-flash (OpenCode Go), gemma-4-26b-a4b-it (Google Gemini) |
| LLM judge          | nemotron-3-super:cloud (Ollama Cloud, temperature=0)                                                                                      |
| Chunk size         | 1,500 chars (closing threshold, not fixed)                                                                                                |
| Overlap            | 150 chars                                                                                                                                 |
| Min chunk length   | 40 chars                                                                                                                                  |
| Top-K retrieval    | 3                                                                                                                                         |
| Temperature        | 0.1                                                                                                                                       |
| Max tokens         | 256 (1,024 for DeepSeek V4 Flash)                                                                                                         |
| Shallow threshold  | 200 informative characters                                                                                                                |
| Scoring            | Fuzzy word-overlap (threshold 0.4)                                                                                                        |
| Bootstrap          | 10,000 resamples, SEED = 42                                                                                                               |
| Collection names   | rag_papers_raw, rag_papers, rag_papers_md_filtered<br>(+ embedder suffix for robustness checks)                                           |
| Dates of execution | May 12–13, 2026                                                                                                                           |
| Code               | <a href="https://github.com/cioffiAI/rag-vs-markdown">https://github.com/cioffiAI/rag-vs-markdown</a>                                     |

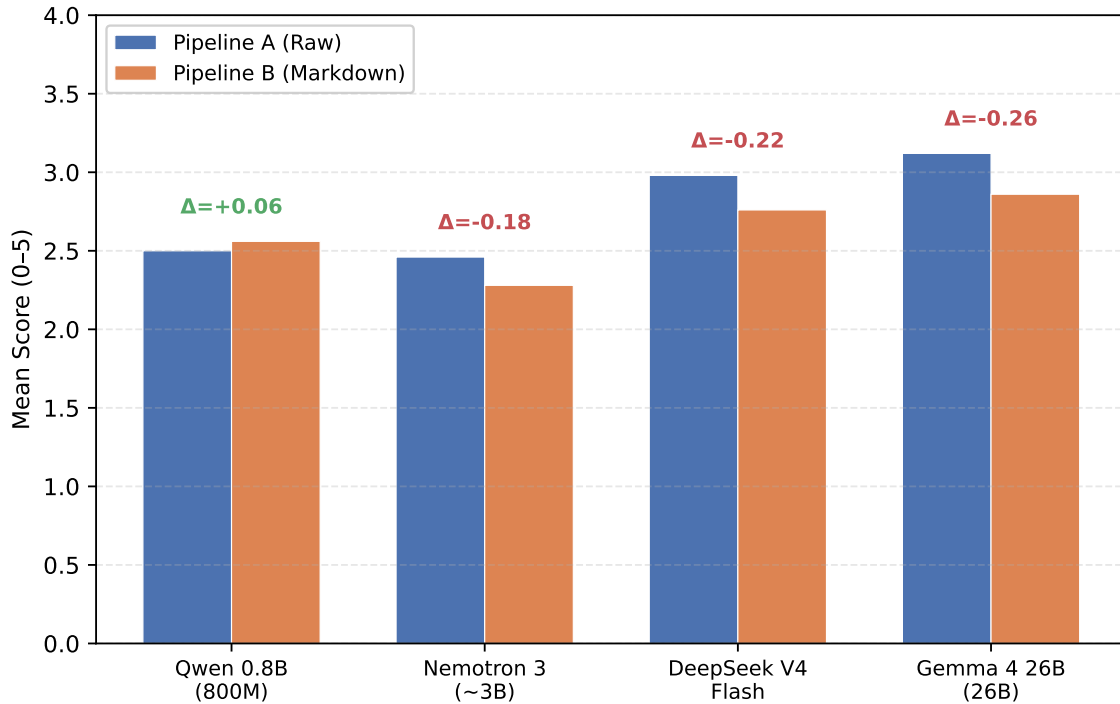


Figure 2: Mean score per model and pipeline (50 questions, base corpus).  $\Delta$  values annotated above each pair. All four models favor Raw over Markdown. Pipeline C (MD-filtered) partially recovers for three models.



Figure 3: Pipeline B – A delta vs. model size (log scale). All four models show a negative delta; the shallow chunk filter (Pipeline C) partially recovers toward Raw. DeepSeek V4 Flash size undisclosed (shown at 10B for visualization).

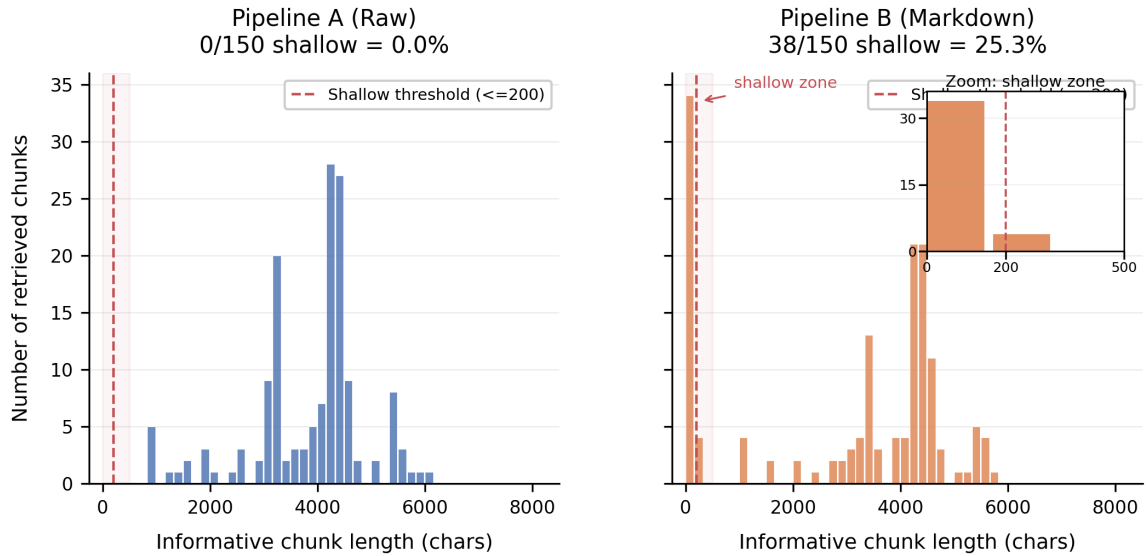


Figure 4: Distribution of retrieved chunk lengths (informative text, Markdown syntax stripped). Pipeline A (Raw) has no chunks  $\leq 200$  characters. Pipeline B (Markdown) has 38/150 (25.3%) shallow chunks; inset zooms on the 0–500 range.

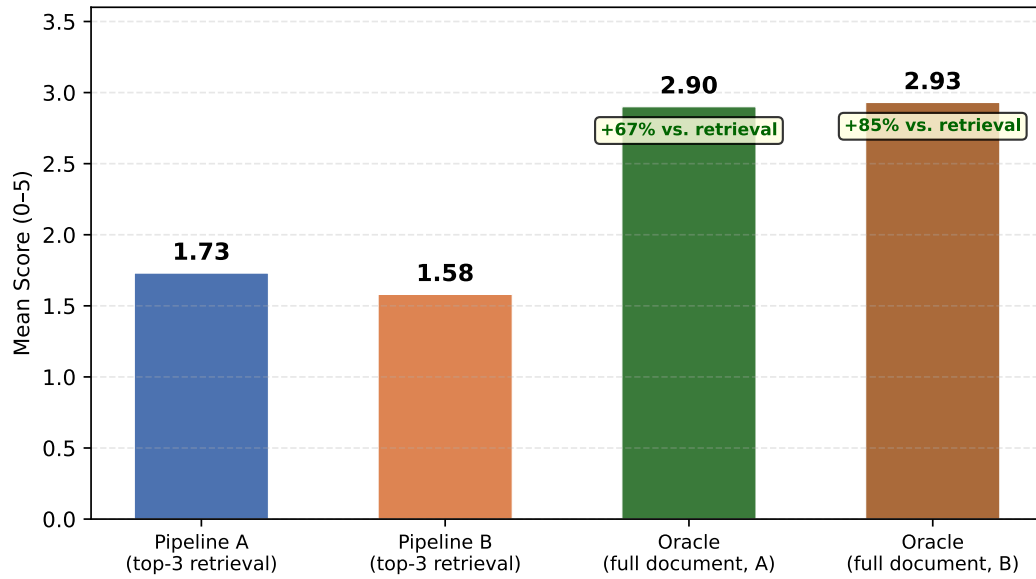


Figure 5: Oracle test: retrieval (top-3 chunks) vs. full-document context. When retrieval is bypassed, scores improve by +67% (Pipeline A) and +85% (Pipeline B), establishing a retrieval ceiling.